Threads in Java

07 April 2024 09:22

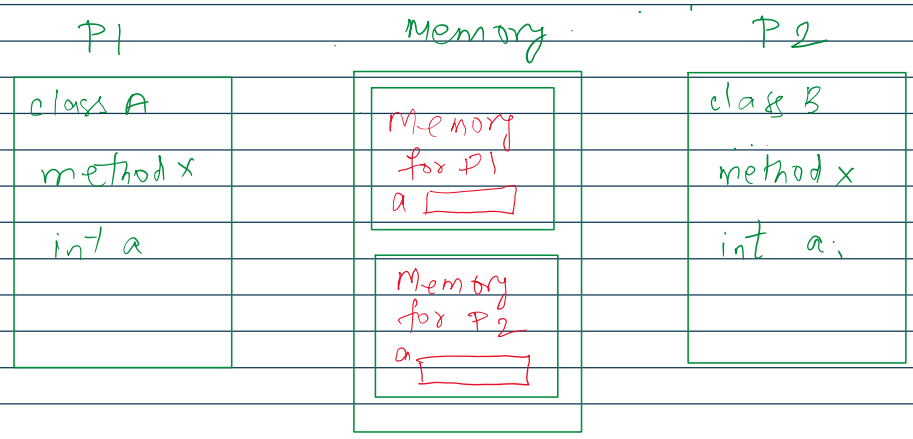
Introduction to threads in Java

Processes versus threads

The operating system is tasked with allocating the necessary resources (memory, processing time, input/outputs to the process and ensuring they do not interfere with one another (isolation).

This principle is illustrated in the following example, in which variable 'a' is defined in two different classes.

When executing both classes, the two allocated memory zone for this variable are completely isolated.

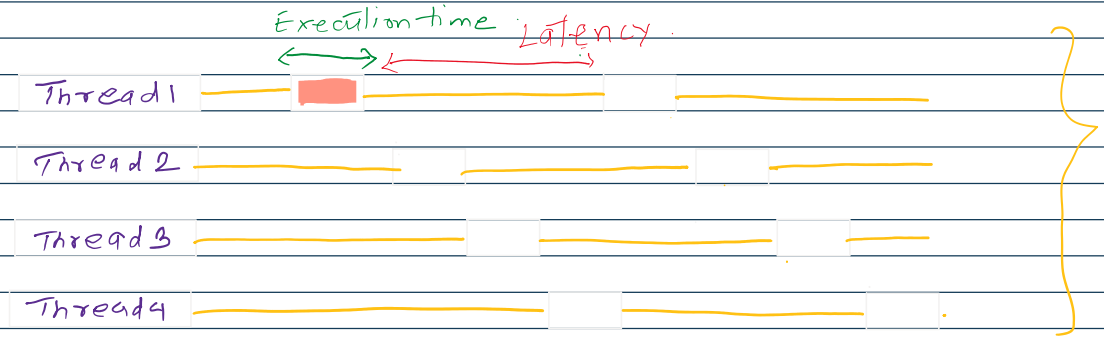


Isolation between Processes.

Most operating system offer a distinction between:

- Heavy-weight processes: supposedly completely separate from one another.
- Light weight processes (threads): which share a memory space (as well as other resources) in common.

Definition: A Thread, is a string of code capable of executing alongside other process. Threads do not execute at same time, but rather using shared time. This is why, it is important that a thread always gives other a chance to execute So let me explain with a diagram:



Execution and latency time of 4 different threads.

Concurrent Computing

The concurrent programming language must allow the following:

- Creation of threads
- Sharing data between threads.
- Synchronization among threats: controlling the task execution order depending on the two. following models:
  - Competition synchronization: when more than one thread is using the same resource there must. then be a system for mutual exclusion in order to avoid process interfering with each other.
  - Cooperation synchronization: when one thread waits for another to finish executing before starting its own execution.

Thread creation

There are two ways to create a thread in Java:

- A class derived from java.lang.Thread:
  - The java.lang.Thread implements Runable class.
  - Thread extends Object implements Runnable public class.
  - The Thread class must implement the run() method.
  - The child class inherits the run() method.
- A class that implements the Runnable interface:
  - The class must implement the run method

- Now, question arises: which solution should we choose?
  - Method-1: subclassing Thread:
    - When parallelizing a class which does not inherit from another class(autonomous class).
    - Note: single inheritance in Java
    - Extends Thread or implements Runnable either way.
  - Method-2: implement Runnable:
    - When a super class is imposed.
    - Example, case involving applets or swing:

```
public class MyThreadApplet extends Applet implements Runnable{
        //your code
}
```

  - Only implements Runnable is valid.

==>Method 1: subclassing Thread

```
class A extends Thread{
    A(){...}  //the constructor
    ...
    @Override
    public void run(){
        .... // what the thread does
    }
}

A t1 = new A(); //Creation of thread t1
t1.start(); //starts the thread and executes t1.run()
```

Thread (Object o) {
this.o = o
}
public void start () {
o.run();
}
↑ is an object of any childclass

===>Method 2: class that implements Runnable

```
class B implements Runnable{
    B(){...} // Constructor
    ...
    @Override
    public void run(){
        .... // what the thread does
    }
}

B p = new B();
```

```
Thread t2 = new Thread(p);

t2.start();   //Start the thread and executes p.run()
```